

Teknologisk Institut

AFSLUTTENDE EKSAMENSPROJEKT

Udvikling og integration af humanoid robot

Forfatter:
Alexander Christensen

Vejleder:
Rasmus Klitgaard

Rapport afleveret som del af uddannelsen til Automationsteknolog AK

ved

UCL ERHVERVSAKADEMI OG
PROFESSIONSHØJSKOLE

15. juni 2026

Indhold

1	Indledning	1
1.1	Baggrund	1
1.2	Problemformulering	2
1.3	Afgrænsning	2
1.4	Målgruppe	2
1.5	Metode	3
1.6	Rapportens opbygning	3
2	Teori og baggrund	4
2.1	Anvendelseskontekst	4
2.2	Seriebus-servoen som integrations-komponent	5
2.3	Kommunikation: USB-serial og ROS 2	5
2.4	3D-print som konstruktionsmateriale	5
3	Analyse af eksisterende system	6
3.1	Robotten som helhed	6
3.2	Arm-delsystemet	6
3.2.1	Aktuatorer	7
3.2.2	Styreelektronik	7
3.2.3	Mekanisk opbygning	7
3.2.4	Signal- og effektkæde	7
3.3	Eksisterende dokumentation og kildekode	8
3.4	Begrundelse for valg af arm som delsystem	8
4	Mekanisk fremstilling og samling	9
4.1	Print-strategi for armens dele	9
4.2	Samling og pasninger	9
4.3	Status og resultat	9
5	Elektrisk integration og wiring	10
5.1	Komponentoversigt	10
5.2	Signalkæde og wiring	10
5.3	Stabil portbinding for ESP32-bokse	11
5.4	Praktiske valg under integrationen	11
6	Idriftsættelse og test	12
6.1	Software-opsætning	12
6.2	PID-konfiguration og kontrolarkitektur	12
6.3	Kalibrering af led	13

6.4	Tilføjelser: commissioning-værktøjer	13
6.5	Funktionstest: step-response pr. led	14
6.6	Hvad værktøjerne måler – og ikke måler	14
6.7	Andre integrationsrettelser	15
6.8	Resultat	16
7	Diskussion	17
7.1	Hvad step-respons-målingerne faktisk siger	17
7.2	Open-loop på ROS-niveau som design-valg	17
7.3	Stabil portbinding	18
7.4	Mekaniske grænser i 3D-printet arm	18
7.5	Påbegyndt, men ikke leveret: per-led-effektmåling	18
7.6	Robusthed i værktøjs-laget	19
8	Konklusion	20
9	Perspektivering	22
9.1	Det videre arbejde på Wattson	22
9.2	Fra prototype mod anvendelse	22
9.3	Humanoide robotter som supplement til klassisk automation	23
A	Bilag	24
A.1	Tabel 1 – Kalibreringsværdier for venstre arm	24
A.2	Tabel 2 – Step-respons-resultater	24
A.3	Tabel 3 – Komponentliste	25
A.4	Figur 1 – Step-respons for skulderens pitch-led	25
A.5	Foto-bilag	26

Kapitel 1

Indledning

1.1 Baggrund

Denne rapport er udarbejdet i forbindelse med praktikforløb hos Teknologisk Institut som del af uddannelsen til automationsteknolog ved UCL Erhvervsakademi og Professionshøjskole. Teknologisk Institut har overtaget et projekt fra Energinet som har startet udvikling af en fuldt 3D-printet humanoid robot under navnet *Wattson*. Robotten består udelukkende af 3D-printede mekaniske komponenter samt standardelektronik (servoer, mikrocontrollere og en Jetson-baseret styreenhed), og den drives af en eksisterende softwarestack baseret på ROS 2. Wattson er oprindeligt udviklet med henblik på vedligehold- og inspektionsopgaver i højspændingsanlæg, hvor en menneskeformet robotmorfologi gør det muligt at færdes i den eksisterende, menneskeskabte infrastruktur uden at miljøet skal ombygges.

Den oprindelige robot blev udviklet over ca. ét år af to personer, og både mekanisk design, elektrisk integration og softwarearkitektur er dokumenteret i et offentligt tilgængeligt git-repository. Da hardwaren er fuldt 3D-printet, er det i princippet muligt at fremstille en kopi af robotten ud fra de eksisterende filer. I praksis kræver det dog en betydelig integrationsindsats, før de printede dele, elektronikken og softwaren udgør et fungerende system.

Min opgave i praktikken har været at gennemføre denne integrationsindsats: at fremstille de mekaniske dele, samle dem, integrere elektronikken, idriftsætte softwaren og dokumentere processen, så resultatet kan anvendes som udgangspunkt for videre udvikling hos Teknologisk Institut.

1.2 Problemformulering

Hvordan kan en humanoid robot, baseret på eksisterende 3D-design og software, fremstilles, samles og integreres til en funktionel enhed, hvor mekaniske og elektriske systemer korrekt understøtter robottens bevægelser?

Projektet omfatter:

- 3D-print og efterbehandling af robottens komponenter
- Mekanisk samling af robottens delsystemer
- Elektrisk opbygning og korrekt wiring af aktuatorer og styresystem
- Integration og afprøvning af eksisterende software

Der arbejdes med at sikre korrekt samspil mellem systemets delkomponenter samt mulighed for tilpasning og optimering af udvalgte komponenter og styring med henblik på at forbedre funktionalitet og stabilitet.

1.3 Afgrænsning

Projektet afgrænses til at fokusere på udvikling, fremstilling og integration af en humanoid robotarm som et selvstændigt delsystem.

Robotarmen er valgt som fokusområde, da den indeholder centrale mekaniske og styringsmæssige udfordringer, herunder bevægelse i flere led, belastning af komponenter samt koordinering af aktuatorer.

Projektet vil derfor ikke nødvendigvis omfatte en fuld færdiggørelse af hele den humanoide robot, men der arbejdes med en modulær tilgang.

Følgende dele af robotten behandles ikke i rapporten:

- Den modsatte arm, torso, hofteparti, ben og hoved
- Teleoperation via VR-udstyr (Quest 3 og ZED-kamera), som det eksisterende softwaresystem understøtter
- Invers kinematik og bevægelsesplanlægning på systemniveau

Hvis dele af det øvrige system er færdige ved eksamenstidspunktet, kan de demonstreres mundtligt, men de udgør ikke en del af den skriftlige bedømmelse.

1.4 Målgruppe

Projektet henvender sig primært til teknikere og fagpersoner inden for automation og robotteknologi, som arbejder med opbygning, integration og idriftsættelse af robotsystemer.

Fokus er på en praktisk og teknisk formidling.

1.5 Metode

Projektet er gennemført som et idriftsættelsesforløb: et eksisterende design omsættes til en fungerende fysisk enhed gennem fremstilling, samling, elektrisk integration og softwarekonfiguration. Arbejdet er foregået iterativt – når et problem opstod under samling eller test, blev årsagen diagnosticeret og en løsning implementeret. De vigtigste af disse hændelser er medtaget i rapporten, fordi de illustrerer de praktiske konsekvenser af at bygge med 3D-printede komponenter og af at integrere et eksisterende, kun delvist dokumenteret softwaresystem.

Ud over selve idriftsættelsen omfatter projektet en række tilføjelser til det oprindelige system, som er udviklet for at gøre det praktisk at arbejde med: stabil portbinding for de tilsluttede mikrocontrollere, en samlet betjenings-GUI til opstart og demo, samt en commissioning-pakke med værktøjer til at måle og dokumentere armens opførsel. Disse tilføjelser er beskrevet i de relevante kapitler sammen med de problemer, der motiverede dem.

1.6 Rapportens opbygning

Rapporten er opbygget som følger:

- **Kapitel 2** – den baggrundsviden, der anvendes senere i rapporten: seriebus-servoer som integrations-komponent, ROS 2 som kommunikationsplatform samt egenskaberne ved 3D-printede konstruktionsdele.
- **Kapitel 3** – det eksisterende system og begrundelse for afgrænsningen til én arm.
- **Kapitel 4** – den mekaniske fremstilling og samling.
- **Kapitel 5** – elektrisk integration og wiring.
- **Kapitel 6** – idriftsættelse, herunder softwareopsætning, kalibrering og test.
- **Kapitel 7** – konsekvenserne af at have bygget armen udelukkende i 3D-printede dele.
- **Kapitel 8** – besvarelse af problemformuleringen.
- **Kapitel 9** – det videre arbejde.

Kapitel 2

Teori og baggrund

2.1 Anvendelseskontekst

Wattson er oprindeligt udviklet med henblik på vedligehold- og inspektionsopgaver i højspændingsanlæg. Denne rapport behandler ikke en konkret højspændingsanvendelse, men anvendelseskonteksten er relevant, fordi den begrunder en række af de design- og komponentvalg, integrationen i de følgende kapitler er underlagt. Konteksten leverer dermed den ramme, hvori dimensionering, wiring og softwareafgrænsning skal forstås.

Højspændingsanlæg er bygget til mennesker. Betjeningspaneler, ventiler, afbrydere og gangbroer er placeret i menneskehøjde og dimensioneret til menneskelig rækkevidde. En klassisk industrirobot på et fast fundament kan derfor kun dække en lille del af et anlæg uden, at miljøet bygges om. En menneskeformet robot kan derimod i princippet færdes og betjene den eksisterende infrastruktur uden tilpasning. Det forklarer valget af en to-armet, opretstående konstruktion med en rækkevidde tæt på en voksen persons og en antropomorf arm med 7 DOF, hvor håndens position og orientering kan nås på flere måder.

Konteksten forklarer også flere af de arkitekturvalg, integrationen er underlagt:

- Hver arm har sin egen serielle servo-bus med dedikeret mikrocontroller, så delsystemer kan idriftsættes og fejlfindes hver for sig – en forudsætning for, at én arm kan bygges og afprøves uafhængigt af resten af robotten.
- Den eksisterende softwarestack understøtter teleoperation via VR-udstyr; det forudsætter pålidelig, lav-latens kommunikation mellem operatør, Jetson og servo-bus, hvilket stiller krav til buslayout og kabelføring, der gælder uanset styringsmodus.
- Mekaniske og elektriske grænseflader er designet til at kunne demonteres uden specialværktøj, så printede slidkomponenter kan udskiftes i drift.

Disse rammebetingelser er fastlagt før integrationsarbejdet, men de er afgørende for at forstå, hvorfor armen ser ud, som den gør, og hvorfor de komponenter, der er bestilt og monteret, er valgt, som de er.

2.2 Seriebus-servoen som integrations-komponent

2.3 Kommunikation: USB-serial og ROS 2

2.4 3D-print som konstruktionsmateriale

Kapitel 3

Analyse af eksisterende system

Før den praktiske integrationsindsats kan beskrives, er det nødvendigt at forstå det system, der danner udgangspunkt for projektet. Dette kapitel gennemgår den oprindelige Wattson-robot på systemniveau, beskriver den arm, der udgør rapportens fokus, og redegør kort for den eksisterende dokumentation og softwarestack. Til sidst begrundes valget af armen som afgrænset delsystem.

3.1 Robotten som helhed

Wattson er en bipedal humanoid robot, hvor samtlige bærende mekaniske dele er fremstillet ved FDM-3D-print. Robotten består overordnet af følgende delsystemer:

- To arme, hver med syv frihedsgrader (skulder, overarm, albue, underarm og håndled)
- To hænder med adskilte finger-led
- Hoved med to frihedsgrader (pitch og yaw)
- Torso, hofte og to ben med flere led pr. ben

Robotten styres af en enkelt indlejret computer (Jetson Orin Nano), som afvikler en eksisterende ROS 2 Humble-baseret softwarestack. Aktuatorerne er fordelt på flere dedikerede mikrocontrollere (ESP32), der hver styrer en logisk gruppe af servoer og kommunikerer med Jetsonen via USB-serial. Den oprindelige robot understøtter teleoperation via et VR-headset og et ZED-stereokamera, men disse funktioner er ikke en del af rapportens scope (jf. kapitel 1).

3.2 Arm-delsystemet

En arm er den enhed, der i rapporten behandles som selvstændigt delsystem. Armen består af syv aktive led, der tilsammen udgør en antropomorf kineamatik: skulder (pitch, roll, yaw), albue (pitch), underarm (yaw) og håndled (pitch, roll).

3.2.1 Aktuatorer

De syv led i armen drives af Waveshare ST3215 seriebus-servoer. Dette er en intelligent servotype med indbygget mikrocontroller, magnetisk positionsencoder og en intern PID-løkke, der lukker positionsreguleringen om sin egen aksel. Servoerne kommunikerer over en halv-duplex TTL-seriel bus, hvor flere servoer er forbundet i daisy-chain og adresseres ved et unikt ID (i denne arm er ID 1–7 anvendt).

Den interne PID-løkke gør, at den overordnede styring kun behøver sende en ønsket position til hver servo; den lokale regulator i servoen sørger selv for at nå den. Reguleringsparametrene (proportional-, integral- og differentiel) er konfigurerbare pr. servo og indlæses fra en JSON-fil ved opstart, jf. afsnit 3.3.

3.2.2 Styreelektronik

Servoerne er ikke forbundet direkte til Jetsonen, men til en dedikeret ESP32-mikrocontroller, der fungerer som bus-master på den serielle servobus. ESP32'en er via USB forbundet til Jetsonen og fremstår dér som en virtuel COM-port ("`/dev/ttyUSBx`"). I den fuldt opbyggede robot er der flere ESP32-controllere (én pr. logisk gruppe – venstre arm + hoved, højre arm, hænder), men i den afgrænsede konfiguration anvendes kun den, der styrer den valgte arm.

3.2.3 Mekanisk opbygning

De mekaniske led består af 3D-printede strukturdele, der monteres direkte på servoernes udgangsaksler. Belastningen i leddet bæres altså af både selve den printede del og af servoens lejer. Denne konstruktion er let og billig, men introducerer nogle særlige hensyn omkring styrke, stivhed og slid, som behandles i kapitel 7.

3.2.4 Signal- og effektkæde

På et overordnet niveau ser signal- og effektkæden for én arm således ud:

- **Signal:** Jetson Orin Nano → USB → ESP32 → TTL-seriel bus → ST3215 ID 1–7.
- **Effekt:** Strømforsyning → fordelingsskinne → ST3215 ID 1–7 (nominel driftsspænding ca. 7,4 V).

3.3 Eksisterende dokumentation og kildekode

Hele Wattson-projektet er offentligt tilgængeligt som et ROS 2-workspace på GitHub. Workspacet indeholder bl.a. pakker til servo-styring, teleoperation, kamera-integration og opstart af robotten. For rapportens formål er især følgende relevant:

- “servos.launch.py” – starter en “wattson_servo_manager_node”, der indlæser servokonfigurationer fra en angiven mappe og opretter ROS-topics for hver gruppe.
- “slider_control.launch.py” – en GUI med skyderkontroller pr. led. Vel egnet til idriftsættelse, fordi den ikke kræver VR-udstyr, kamera eller invers kinematik.
- “servo_arm_left_params.json” – konfigurationsfil for den venstre arms syv servoer. Indeholder for hvert led bl.a. servo-ID, gear-ratio, PID-gains (“gain_P”, “gain_I”, “gain_D”), software-grænser og udgangsposition.

Det er væsentligt, at konfigurationsfilen for en arm er selvstændig. Hver arm kan derfor i princippet køres uafhængigt af resten af robotten, hvilket er en forudsætning for det modulære arbejde i denne rapport.

3.4 Begrundelse for valg af arm som delsystem

Armen er valgt som afgrænset delsystem, fordi den i kompakt form indeholder alle de elementer, der karakteriserer Wattson-projektet som helhed:

- En 3D-printet mekanisk struktur med flere bevægelige led og dynamisk belastning.
- Flere elektriske aktuatorer, der skal forsynes og styres samtidigt.
- En dedikeret mikrocontroller (ESP32), der skal idriftsættes og kommunikere med en overordnet computer.
- En reguleringsmæssig styring – både den lokale PID-løkke i hver servo og den overordnede positionskommando fra ROS.
- En kommunikationskæde i flere niveauer (USB, TTL-serial, servo-protokol).

Armen er samtidig stor nok til at give et meningsfuldt demonstrationsresultat, men lille nok til at kunne fuldføres og dokumenteres inden for projektets tidsramme. Dette delsystem danner udgangspunkt for de følgende kapitler om mekanisk fremstilling (kapitel 4), elektrisk integration (kapitel 5) og idriftsættelse (kapitel 6).

Kapitel 4

Mekanisk fremstilling og samling

Robottens mekaniske dele er fuldt 3D-printede efter Energinets eksisterende konstruktion. Projektet har ikke ændret på det mekaniske design; arbejdet i dette kapitel omfatter fremstilling af en arbejdsklar replica af venstre arm og de tilretninger, der opstod undervejs, når printede dele skulle samles til en fungerende arm.

4.1 Print-strategi for armens dele

4.2 Samling og pasninger

4.3 Status og resultat

Kapitel 5

Elektrisk integration og wiring

Dette kapitel beskriver den elektriske opbygning af det arm-delsystem, projektet omfatter: hvilke komponenter der indgår, hvordan de er forbundet, og hvilke integrationsproblemer der opstod undervejs.

5.1 Komponentoversigt

Det elektriske delsystem omfatter komponenter på tre niveauer: aktuatorer i armen, en kommunikations-mikrocontroller mellem arm og host, og en host, der kører ROS 2-laget. Den fulde komponentliste fremgår af Tabel 5.1. Farvemærkningen i tabellen følger den fysiske kabel-isolering på de tre ESP32-bokse, så hver USB-port på hubben permanent svarer til den samme logiske funktion.

TABEL 5.1: Hardware-komponenter for venstre arm.

Komponent	Antal	Kommentar
Waveshare ST3215-servo	7	12 V, ID 1–7.
Waveshare SC09-servo	5	5 V, håndens fem fingre.
ESP32-baseret USB-bridge	1	USB til servobus.
Mean Well DDR-120	1	DC-DC-omformer, 12 V til ST3215.
Chinfa DRD15 13.5W	1	DC-DC-omformer, 5 V til SC09.
RS Pro 264V 120W	1	Hovedforsyning, 120 W.
Siemens Sentron 8A	2	Sikringer på 12 V- og 5 V-skinnen.
USB-hub med 3 porte	1	Rød, gul, hvid.
Jetson Orin Nano	1	ROS 2-host.

5.2 Signalkæde og wiring

Signalvejen i den oprindelige konstruktion er kort og lineær:

1. Jetson Orin Nano sender positionskommandoer på et ROS-topic.

2. Manager-noden serialiserer kommandoerne og skriver dem på en virtuel COM-port mod ESP32-boksen.
3. ESP32-boksen konverterer beskeden til servobus-protokollen og videresender den på den halv-duplexerede TTL-bus.
4. Hver ST3215-servo på bussen filtrerer på sit servo-ID og udfører den modtagne positionskommando i sin egen interne PID-sløjfe.

Effektvejen er adskilt fra signalvejen: servobussen forsynes direkte fra sin egen forsyning, mens ESP32 og logik-niveau strømmes via USB fra Jetson'en.

5.3 Stabil portbinding for ESP32-bokse

Den væsentligste elektriske/integrationsmæssige rettelse i projektet vedrører tildelingen af USB-porte til de tre ESP32-bokse. Tidligere blev de adresseret som `"/dev/ttyUSB0"`, `"/dev/ttyUSB1"` og `"/dev/ttyUSB2"`, og rækkefølgen afhang af, hvilken boks operativsystemet enumererede først ved boot. I praksis betød det, at manager-noden kunne starte med at sende den venstre arms kommandoer til den højre arms ESP32, hvis kablerne var sat i en anden rækkefølge. Systemet kører, det kører bare mod den forkerte hardware.

Løsningen er at adressere boksene via deres faste sti i `"/dev/serial/by-path/"` frem for en bestemt tilslutningsrækkefølge `"/dev/ttyUSBx"`. Hver fysisk USB-port på Jetson-hubben har en unik sti, der ikke ændrer sig mellem boots eller hvis maskinen genstartes. Manager-nodens konfiguration blev opdateret til at slå op i denne sti, og der blev oprettet dokumentation for hvor hver ESP32 sluttes til.

- Port 2.2 – RØDT kabel – venstre arm
- Port 2.3 – GULT kabel – højre arm og hoved
- Port 2.1 – HVIDT kabel – hænder

Mappingen afhænger nu af, hvilken fysisk USB-port boksen sidder i, ikke af om operatøren husker rækkefølgen ved tilslutning.

5.4 Praktiske valg under integrationen

Kapitel 6

Idriftsættelse og test

Dette kapitel dokumenterer den software- og styringsmæssige idriftsættelse af armen efter den mekaniske og elektriske integration. Kapitlet beskriver i hvilken konfiguration robotten faktisk køres, hvilke værktøjer der er udviklet for at kunne måle armens opførsel, hvad disse værktøjer faktisk måler – og lige så vigtigt: hvad de *ikke* måler – samt en række konkrete problemer, der er stødt på undervejs, og hvordan de er løst.

6.1 Software-opsætning

Idriftsættelsen tager udgangspunkt i Energinets eksisterende ROS 2-workspace (“energirobotter-ros-workspace”), der ligger på Jetson Orin Nano under ROS 2 Humble. For at kunne arbejde isoleret med én arm uden at indlæse konfigurationer for hænder, hoved og højre arm, indlæses kun den relevante “servo_arm_left_params.json”-fil ved opstart af “wattson_servo_manager_node”. Selve opstarten sker via to launch-filer: “servos.launch.py” starter manager-noden, og “slider_control.launch.py” starter en GUI med en skyder pr. led, der gør det muligt at bevæge hvert led uafhængigt.

6.2 PID-konfiguration og kontrolarkitektur

Hver ST3215-servo har en intern positionsregulator. PID-værdierne pr. servo står i JSON-filen og er anvendt uændret fra Energinets konfiguration. Manager-noden videregiver målpositioner til hver servo, og selve reguleringsløjen lukkes inde i den enkelte ST3215.

ROS-laget regulerer ikke selv og modtager ingen tilbagemelding fra servos faktiske position. Som test blev “feedback_enabled” sat til “true” i både JSON-konfig og driverkode for at undersøge, om feedback-vejen kunne aktiveres uden yderligere arbejde. Resultatet var, at alle arm-led begyndte at oscillere, også uden eksterne kommandoer. Open-loop-konfigurationen blev derfor reetableret, og feedback-vejen blev i stedet brugt diagnostisk (afsnit 6.6).

6.3 Kalibrering af led

Inden armen kan styres meningsfuldt, skal hvert led kalibreres. For hver servo indeholder JSON-konfigurationen tre værdier, der definerer leddets arbejdsrum:

- “angle_software_min” og “angle_software_max” – de software-grænser, manager-noden klipper kommandoer til, så hverken servo eller mekanik ødelægges af en ude-af-grænse-kommando.
- “default_position” – den vinkel, leddet sættes til ved opstart.
- “gear_ratio” – omsætningsforholdet mellem servo-aksel og fysisk led, så ROS-niveauet kan tænke i led-vinkler frem for servo-aksel-vinkler.

Kalibreringen er sket praktisk: armen bevæges manuelt med slider-GUI'en, og grænserne sættes til de positioner, hvor leddet fysisk når sit endestop uden at trække i mekanikken. Nul-positionen er sat til en geometrisk neutral pose, hvor armen hænger lodret langs torsoens side. Værdierne for venstre arm er dokumenteret i bilaget (se *Bilag, Tabel 1*).

6.4 Tilføjelser: commissioning-værktøjer

Det oprindelige system har ingen indbyggede værktøjer til at måle eller dokumentere armens dynamiske opførsel. Til brug for idriftsættelsen er der derfor udviklet en ny ROS-pakke, “arm_commissioning”, der indeholder følgende noder:

- **“step_response_node”** Kommanderer en step-input på en valgt led, logger den feedback-vinkel, manager-noden publicerer, og beregner klassiske step-respons-mål: rise time (10 %–90 %), overshoot, settling time (med konfigurerbar tolerance) og steady-state-fejl. Output: CSV-rådata og en PNG-graf pr. kørsel.
- **“repeatability_node”** Sender armen frem og tilbage mellem to konfigurerede positioner et antal cyklusser og registrerer slut-positionen i hvert hold-vindue. Output: histogram for de to målte pose-populationer plus mean, std og max-afvigelse.
- **“launcher_gui_node”** Et Tk-baseret betjeningspanel, der samler de typiske demo-services (DHCP, kameraets ROS-launch, VR-broer, servo-manager, animationsafspilning) bag start/stop-knapper med statusindikatorer og samlede logfaner. Begrundelse: en eksamensdemo, hvor 5–6 terminal-vinduer skal åbnes i en præcis rækkefølge, er fragil; et samlet panel reducerer antallet af fejlmuligheder.

For at “step_response_node” og “repeatability_node” kunne hente positions-data, blev der desuden tilføjet en feedback-publisher til “wattson_servo_manager_node”: et nyt topic “/joint_states_feedback”, der publicerer den positionsværdi, manager-noden kender for hver servo. Dette topic er den eneste nye afhængighed, commissioning-værktøjerne tilføjer til det eksisterende system.

6.5 Funktionstest: step-response pr. led

Step-response køres på hvert af de syv venstre arm-led med samme parametre: 10° step fra nul, 12 s logging, 50 Hz samplerate, settling-tolerance 10 % af step-amplituden.

Resultatet pr. led sammenfattes i tabellen i bilaget (se *Bilag, Tabel 2*). Et repræsentativt forløb for “joint_left_shoulder_pitch” vises i *Bilag, Figur 1*: den kommanderede vinkel som step-funktion, den loggede feedback-vinkel og de beregnede mål annoteret på plottet.

Afvisninger mellem leddene diskuteres i kapitel 7.

6.6 Hvad værktøjerne måler – og ikke måler

Den feedback-vinkel, “step_response_node” logger, kommer fra ROS-topic’et “/joint_states_feedback”. Det topic publiceres af manager-noden og indeholder i den nuværende konfiguration den *senest kommanderede* vinkel for hvert led, ikke en målt position fra servoens encoder. Det skyldes en tre-niveau gating på flaget “feedback_enabled”, der i den oprindelige konfiguration står til “false”:

1. I JSON-konfigurationen for venstre arm er “feedback_enabled” sat til “false” for alle syv servoer.
2. Driveren “DriverWaveshare” har samme værdi som instans-default, og manager-noden overskriver den ikke.
3. I “ServoControl.compute_command” medfører “feedback_enabled = false”, at den cachede position “self.angle” sættes lig den senest kommanderede vinkel frem for en målt værdi.

Konsekvensen for commissioning-værktøjerne er, at de tal, de udregner, ikke karakteriserer ST3215-servoens interne regulator. Det de måler, er kommandoernes vej gennem systemet – fra en ny target publiceres på “/joint_states”, til den cachede position afspejler den nye target. Det er en nyttig størrelse, fordi den dækker latency i ROS-laget, men det er *ikke* en servo-karakteristik.

Som beskrevet under PID-konfigurationen blev det afprøvet at slå “feedback_enabled” til; armen begyndte at oscillere, og open-loop-konfigurationen blev derfor bibeholdt. Step-respons-tabellen i bilaget skal læses i lyset af dette: tallene er gyldige som sammenligningsgrundlag mellem leddene og som karakteristisk af command-pipelinen, men de er ikke et udtryk for ST3215-regulatorens dynamik.

6.7 Andre integrationsrettelser

Ud over de fund, der direkte vedrører funktionstesten, blev der under idriftsættelsen rettet en række mindre integrationsfejl. De vigtigste er listet nedenfor; detaljer ligger i kildekodens commit-historik og bilag.

Stabil portbinding for ESP32-bokse. Tidligere blev de tre USB-tilsluttede mikrocontrollere adresseret som `"/dev/ttyUSB0-2"`, hvilket gjorde tildeelingen afhængig af tilslutningsrækkefølge. Manager-noden adresserer dem nu via `"/dev/serial/by-path/"`, så hver fysisk port på Jetson-USB-hubben permanent svarer til den samme logiske funktion. Detaljer i kapitel 5.

Race-tilstand i kinematik-nodens opstart. `"elrik_kdl_kinematics_node"` crashe sporadisk under opstart med en `"AttributeError"`, fordi en periodisk timer blev oprettet, før node-instansens egne felter var initialiseret. Fixet ved at flytte `"create_timer"`-kaldet til slutningen af `"__init__"`, så timeren først aktiveres, når nodens tilstand er klar.

Launcher-GUI bugs under hardware-rehearsal. Det samlede betjeningspanel blev iterativt rettet for en række problemer, der først kom frem under faktisk demo-brug. De fire vigtigste rettelser beskrives nedenfor.

Adgangskode-prompts ved fjernlogin. Launches af ROS-noder på Jetson sker fra GUI'en via en krypteret fjernforbindelse. Oprindeligt skulle operatøren indtaste password ved hvert kald, hvilket gjorde et sammenhængende demo-flow uigennemførligt. Problemet blev løst med to mekanismer. Først blev der lavet en lille hjælpe-proces, som leverer adgangskoden gennem et grafisk dialogvindue, så fjernforbindelsen ikke længere kræver, at GUI'en har en terminal at spørge i. Dernæst blev der konfigureret deling af forbindelsen, så det første login etablerer en baggrunds-forbindelse, som efterfølgende kald genbruger. Resultatet er, at password kun spørges én gang pr. service og ikke én gang pr. ROS-launch. En mere permanent løsning ville være login med kryptografisk nøgle frem for adgangskode, men det er fravalgt, da det ville kræve ændringer i Jetsons konfiguration uden for projektets afgrænsning.

Langsom respons på stop-knappen. Den oprindelige stop-knap i GUI'en reagerede med 30–60 sekunders forsinkelse, hvilket gjorde nødstop praktisk ubrugeligt. Årsagen var, at stop-signalet ikke nåede helt ned til de ROS-noder, der faktisk kørte: signalet blev fanget af en mellemliggende skal-proces, og selve noden levede videre som en frikoblet baggrunds-proces. Rettelsen bestod i tre dele. For det første blev kommandoen omskrevet, så ROS-noden overtager skal-processen direkte i stedet for at køre som dens barn. For det andet startes hver service nu i sin egen proces-gruppe, så stop-knappen kan signalere hele gruppen på én gang og dermed også fange middleware- og controller-processer,

der eventuelt er startet sammen med noden. For det tredje sendes først et blødt afbrydelses-signal, som lader noden rydde op, og hvis processen mod forventning stadig kører efter en kort frist, sendes et hårdere terminerings-signal som fallback. Tilsammen bringer ændringerne responstiden under et sekund.

Manglende synkronisering mellem laptop og Jetson. Under en re-faktorering blev kildekoden til en central node på laptop-siden uforvarende beskadiget, men på Jetson kørte systemet tilsyneladende videre uden fejl. Forklaringen var, at Python på Jetson havde en cached udgave af samme modul liggende fra en tidligere build og foretrak den frem for at fejle på den nye kilde. Diskrepansen blev først opdaget, da Jetson blev rebuildt fra grunden og pludselig fejlede ved opstart. Den varige rettelse blev derfor proces-mæssig og ikke kode-mæssig: før en kritisk demo skal Jetson altid bygges fra rene mapper, så det testede svarer til det kørende.

Øvrige mindre rettelser. Ved luk af GUI'en kunne et stop-kald ramme en system-tjeneste ejet af root og resultere i en rettighedsfejl, så vinduet hang. Det blev rettet ved at omslutte alle service-stop i fejlhåndtering og foretage netop dette stop med forhøjede rettigheder. Dernæst blev et eksperiment med at hæve kontrol-frekvensen fra 10 til 50 Hz for jævne bevægelse rullet tilbage igen. Robottens 14 servoer deler en fælles seriebus, og målingerne pr. servo er begrænset af bussens gennemløb, ikke af CPU'en. En cyklustid på 20 ms svarende til 50 Hz er derfor fysisk umulig på den eksisterende bus, og forsøget gav i praksis ustabile bevægelser uden gevinst.

6.8 Resultat

Ved afleveringstidspunktet kan armen styres positionsmæssigt led for led med slider-GUI'en og kan afspille præindspillede animations-sekvenser fra commissioning-pakkens animations-fane. Pålideligheden af de underliggende ROS-topics er verificeret via "ros2 topic hz".

Step-respons køres på alle syv venstre arm-led og dokumenteres i bilaget.

De væsentligste ikke-opnåede mål er en kalibreret closed-loop positionsregulering på ROS-niveau (fravalgt, jf. ovenfor) og en funktionsdygtig per-led-effektmåling; sidstnævnte er påbegyndt, men ikke leveret, og er diskuteret i kapitel 7.

Kapitel 7

Diskussion

Dette kapitel diskuterer betydningen af de fund og de valg, der er beskrevet i kapitlerne om mekanik, elektrisk integration og idriftsættelse. Fokus er på, hvad resultaterne kan og ikke kan bruges til, hvilke kompromiser projektet har truffet, og hvad der er påbegyndt men ikke leveret.

7.1 Hvad step-respons-målingerne faktisk siger

Som beskrevet i afsnit 6.6 er den feedback-vinkel, commissioning-værktøjerne logger, identisk med den senest kommanderede vinkel for det pågældende led. Det er ikke en målt position fra servoens encoder. Konsekvensen er, at de rise time-, overshoot- og settling time-tal, der vil komme i bilaget efter lab-kørslerne, ikke karakteriserer ST3215-servoens interne regulator – de karakteriserer *kommando-vejen* fra GUI'en gennem ROS-topics og manager-noden, indtil den cachede position opdateres.

Det betyder ikke, at tallene er værdiløse. De er meningsfulde som sammenligningsgrundlag mellem leddene: hvis tre led har næsten ens target-pipelines og ét led skiller sig ud, peger det på, at noget i manager-loopet eller i kommandobatchningen behandler det led anderledes. Men de kan ikke bruges til at sige noget om servoens mekaniske dynamik, dens evne til at holde en position under last, eller dens reelle settling-opførsel.

7.2 Open-loop på ROS-niveau som design-valg

ROS-laget videresender bare positionskommandoer ud til servoerne og laver ikke nogen regulering selv. ST3215-servoen kører sin egen PID-sløjfe internt, og PID-gainsne i JSON-konfigurationen er Energinets standardværdier. Da "feedback_enabled" blev sat til "true" som test, blev armen øjeblikkeligt ustabil, og open-loop-konfigurationen blev derfor reetableret.

Konsekvensen for projektet er, at en kalibreret closed-loop positionsregulering på ROS-niveau ikke er leveret. Funktioner, der *forudsætter* faktisk positions-feedback – fx en per-led-effektmåling, som er diskuteret nedenfor – kan derfor heller ikke leveres i den nuværende konfiguration.

7.3 Stabil portbinding

Tildelingen af USB-portene til de tre ESP32-bokse afhang tidligere af, hvilken rækkefølge boksene blev sat i, og hvilken rækkefølge operativsystemet enumererede dem i ved boot. Hvis kablerne blev sat i en anden rækkefølge end sidste gang, kunne manager-noden ende med at sende den venstre arms kommandoer til den højre arms ESP32 – uden at det gav nogen åbenlys fejl, fordi systemet *kørte*, det kørte bare mod den forkerte arm.

Med løsningen via `"/dev/serial/by-path/"` afhænger mappingen nu af, hvilken fysisk USB-port boksen sidder i, og ikke af om operatøren husker rækkefølgen. Det er en billig rettelse, der fjerner en hel klasse af fejl, som ellers først dukker op under demo eller efter en flytning. Detaljerne er dokumenteret i kapitel 5.

7.4 Mekaniske grænser i 3D-printet arm

Armens funktionelle arbejdsrum er mindre end dens geometriske rækkevidde. I udstrakt pose med skulderen omkring 90° og albuen strakt observeres, at armen langsomt synker, selv når positionskommandoen holdes konstant. Dette er ikke en software-fejl: gravitationsmomentet på en fuldt udstrakt arm er stort, og når ST3215-servoen ikke længere kan holde imod, synker leddet.

Konsekvensen er, at præindspillede animations-sekvenser og demoer skal holdes inden for poser, hvor det statiske moment er moderat. Det er en konkret begrænsning, der ikke kommer fra software-laget, men fra materialevalget og servovalget i den oprindelige konstruktion.

7.5 Påbegyndt, men ikke leveret: per-led-effektmåling

Som del af commissioning-pakken blev der påbegyndt et værktøj til at måle og logge effektforbruget pr. led. Formålet var at dokumentere termisk belastning og strømforbrug under forskellige bevægelsesmønstre (armen i hvile, teleoperation i neutral pose, teleoperation med strakt arm samt animationsafspilning) som understøttende data til diskussionen i denne rapport. Servoerne stiller selv spændings- og strømaflæsninger til rådighed via servobussen, så det kræver ikke ekstra hardware – alene en udvidelse af software-laget.

Værktøjet blev imidlertid fravalgt inden eksamenstesten. Aflæsningen ville lægge sig oven i den eksisterende trafik på den samme servobus, som armens positionskommandoer i forvejen bruger, og bussens gennemløb er allerede en flaskehals ved den valgte kontrolfrekvens. Den ekstra trafik forskød timingen så meget, at servoernes interne regulator begyndte at oscillere – den samme observation som beskrevet under PID-konfigurationen i kapitel 6. Tradeoff'et stod altså mellem en stabil arm uden effektmåling og en effektmåling med ustabil arm, og det første blev valgt af demonstrationshensyn.

En realistisk videreførelse ville være at flytte aflæsningen ud i en langsommere baggrunds-måling i en separat tråd, så den ikke længere konkurrerer med kontrolloopets tidsbudget på bussen. Det er ikke implementeret i denne aflevering.

7.6 Robusthed i værktøjs-laget

Launcher-GUI'en er ikke en kerne-komponent i robotten, men en integrations-flade mellem operatør og ROS-stak. De rettelser, den krævede under idrift-sættelsen, var derfor primært integrations-rettelser: at SSH-login kun spørger om password én gang og ikke ved hvert kald; at stop-signaler bliver sendt korrekt ned i procestræet for de launch-filer, GUI'en starter; og en kontrol af, at den binær, Jetson'en faktisk kører, matcher den sidste committede kilde-kode (et tidligere mismatch mellem cached binær og kildetræ var årsag til en svær fejl at finde).

Disse rettelser fandtes ved at køre den fulde start-stop-sekvens ende-til-ende på den faktiske hardware. Det er den afprøvningsform, der har været brugbar for launcher-GUI'en; enkeltkomponent-tests af de underliggende ROS-services siger ikke noget om, hvordan GUI'en starter og stopper dem under demobetingelser. Det er en del af baggrunden for at investere tid i et samlet betjeningspanel og en pre-flight-checkliste frem for at lade hver service starte i sit eget terminalvindue.

Kapitel 8

Konklusion

Projektet stillede spørgsmålet om, hvordan en humanoid robotarm baseret på Energinets eksisterende 3D-design og software kan fremstilles, samles og integreres til en funktionel enhed med korrekt samspil mellem de mekaniske og elektriske delsystemer. Konklusionen samler de svar, projektet har leveret, kapitel for kapitel.

Mekanisk fremstilling og samling. Venstre arm er printet, efterbearbejdet og samlet til en arbejdsklar enhed. De samlingsproblemer, der opstod undervejs – snævre servohuse, akselpasninger med slør, kabelgennemføringer – er løst ved manuel efterbearbejdning, og armen bevæger sig mekanisk frit gennem hele det konfigurerede arbejdsrum. Detaljer i kapitel 4.

Elektrisk integration. Armen er forbundet til en ESP32-boks og til Jetson Orin Nano-hosten via stabil portbinding i `"/dev/serial/by-path/"`. Den tidligere rækkefølgeafhængighed i `"/dev/ttyUSBx"` er dermed fjernet som fejlkilde. Signal- og effektveje er adskilte, og montagen er færdig og mærket. Detaljer i kapitel 5.

Software-integration og styring. Armen kan styres positions-mæssigt led for led via slider-GUI'en og kan afspille præindspillede animations-sekvenser. Step-respons køres på alle syv venstre arm-led, og resultaterne dokumenteres i bilaget med det forbehold, der er beskrevet i afsnit 6.6: målingerne karakteriserer command-pipelinen, ikke ST3215-servoens lukkede regulator, fordi ROS-laget bare videresender målpositioner uden at regulere selv. Detaljer i kapitel 6.

Tilføjelser til det oprindelige system. Som del af projektet er der udviklet en commissioning-pakke med to målerelaterede noder (`"step_response_node"` og `"repeatability_node"`) og en samlet betjenings-GUI til opstart og demo. Disse er beskrevet i kapitel 6. En per-led-effektmåling blev påbegyndt, men er ikke leveret i fungerende stand på grund af samme feedback-konfiguration, og den er taget ud af demoen for at undgå at love en måling, brugeren ikke får.

Samlet vurdering. Problemformuleringens kerne – at fremstille, samle og integrere armen til en funktionel enhed med korrekt samspil mellem mekanik og elektrik – er besvaret. Armen fungerer, kan styres, og de integrationsfejl,

der blev fundet undervejs, er enten løst eller eksplicit dokumenteret som vilkår, der følger af den oprindelige arkitektur. De tilføjelser, projektet har bidraget med ud over selve idriftsættelsen, gør det muligt at måle og demonstrere armens opførsel og at starte den samlede system-stak pålideligt.

Kapitel 9

Perspektivering

Rapporten har behandlet integrationen af én arm. Dette kapitel skitserer de naturlige næste skridt for projektet og placerer arbejdet i forhold til den bredere anvendelse af humanoide robotter i automationssammenhæng.

9.1 Det videre arbejde på Wattson

Det mest umiddelbare næste skridt er at udvide integrationen til de øvrige delsystemer i robotten:

- Den modsatte arm, torso, hofteparti og hoved, hvor den modulære arkitektur fra arm-systemet kan genbruges direkte.
- Idriftsættelse af den fulde teleoperation via VR-udstyr og ZED-kamera, som den eksisterende softwarestak allerede understøtter, men som ligger uden for det her projekts afgrænsning.
- Koordineret to-arms-bevægelse, der introducerer afhængigheder på tværs af busser og dermed nye krav til synkronisering og sikkerhedslogik.

Disse skridt udvider robotten i bredden, men ikke væsentligt i kompleksitet pr. delsystem; de metoder, der er anvendt i denne rapport, bør være direkte anvendelige.

9.2 Fra prototype mod anvendelse

Hvis robotten på sigt skal forlade laboratoriet og anvendes i et reelt miljø, ændrer billedet sig væsentligt. Tre forhold trænger sig på:

- **Holdbarhed.** En fuldt 3D-printet konstruktion er velegnet til en prototype, men kryb, slid og temperaturfølsomhed (jf. kapitel 7) gør den uegnet til vedvarende drift uden et planlagt udskiftningsregime eller en delvis ombygning til metal-komponenter i de mest belastede led.
- **Sikkerhed.** I et laboratorium opererer robotten i et indhegnet område. I et anlæg med mennesker omkring sig vil maskindirektivet og standarder for samarbejdende robotter (eksempelvis ISO/TS 15066) stille krav

til kraftbegrænsning, nødstop og risikoanalyse, der ligger ud over det, en hobby-klasse servo-bus understøtter direkte.

- **Pålidelighed under last.** De begrænsninger ved de valgte servoer, der er identificeret i kapitel 7 – manglende detaljeret telemetri og hobby-klassens belastningsgrænser – skal håndteres, før robotten kan udføre opgaver uden konstant tilsyn.

9.3 Humanoide robotter som supplement til klassisk automation

Wattson er en del af en bredere tendens, hvor humanoide robotter undersøges som alternativ – eller snarere som supplement – til klassiske robotløsninger. To observationer er relevante her.

For det første erstatter en humanoid robot ikke en industrirobot på de opgaver, hvor industrirobotten i forvejen er stærkest: høj volumen, høj præcision, fast cyklus og stiv montering. Det er fortsat den klassiske automations domæne, og erfaringerne fra dette projekt – hvor mekanisk repeterbarhed og holdbarhed er begrænset af 3D-printet konstruktion – understreger den grænse.

For det andet adresserer den humanoide form en kategori af opgaver, som klassisk automation har vanskeligt ved: lav volumen, høj variation, eksisterende menneskeskabt infrastruktur og opgaver, hvor end-effector og arbejdspose skifter ofte. Vedligehold og inspektion i højspændingsanlæg, som er Wattsons oprindelige anvendelseskontekst (jf. afsnit 2.1), falder netop i den kategori.

Om humanoide robotter får en udbredt rolle i industrien afhænger derfor mindre af, om de kan slå industrirobotter på deres hjemmebane, og mere af, om de kan løse de opgaver, som industrirobotter i dag ikke håndterer. Erfaringerne fra dette projekt – både hvad der virker, og hvor mekanikken og elektronikken kommer til kort – er et lille bidrag til den vurdering.

Bilag A

Bilag

Dette bilag samler de tabeller, fotos og målegrafer, der er henvist til fra hovedteksten. Tabellerne er nummereret i den rækkefølge, de introduceres i rapporten.

A.1 Tabel 1 – Kalibreringsværdier for venstre arm

Tabel A.1 viser de kalibreringsværdier, der bruges ved opstart af manager-noden for venstre arm. Alle vinkler er angivet i grader. Kolonnen **Retning** angiver, om led og servo drejer samme vej (+) eller modsat (−). **Gear** er antallet af servo-omdrejninger pr. led-omdrejning. **Min** og **Maks** er de softwaregrænser, manager-noden klipper kommandoer til, **Default** er leddets opstartsvinkel, og **Maks. hast.** er den højeste tilladte vinkelhastighed i grader pr. sekund.

TABEL A.1: Kalibreringsværdier for de syv led i venstre arm.
Den indbyggede positions-regulator i hver servo bruges direkte; ROS-laget regulerer ikke selv (jf. kapitel 6).

Led	ID	Retning	Gear	Min	Maks	Default
Skulder, pitch	7	+	3	120	240	150
Skulder, roll	6	−	3	120	230	180
Overarm, yaw	5	+	3	120	230	180
Albue, pitch	4	+	3	120	240	120
Underarm, yaw	3	+	1	120	270	180
Håndled, pitch	2	+	1	120	230	180
Håndled, roll	1	−	1	120	230	180

A.2 Tabel 2 – Step-respons-resultater

Step-respons er målt for hvert led ved at sende et fast vinkel-step fra leddets default-position og logge den feedback-vinkel, manager-noden publicerer (jf. kapitel 6). *Stigetid* er tiden, det tager den loggede vinkel at gå fra 10 % til 90 % af step-amplituden. *Overshoot* er den procentvise overskridelse

af målpositionen i forhold til step-amplituden. *Indsvingningstid* er det tidspunkt, hvor vinklen første gang er inden for $\pm 2\%$ af slutværdien og forbliver der. Resultaterne er sammenfattet i Tabel A.2.

TABEL A.2: Step-respons-metrikker pr. led målt under idrift-sættelsen. Råddata er gemt som CSV-filer, der ledsager rapporten.

Led	Stigetid [s]	Overshoot [%]	Indsvingning [s]
Skulder, pitch	TODO	TODO	TODO
Skulder, roll	TODO	TODO	TODO
Overarm, yaw	TODO	TODO	TODO
Albue, pitch	TODO	TODO	TODO
Underarm, yaw	TODO	TODO	TODO
Håndled, pitch	TODO	TODO	TODO
Håndled, roll	TODO	TODO	TODO

A.3 Tabel 3 – Komponentliste

Den fulde komponentliste fremgår af kapitel 5 (Tabel 5.1) og gengives her som hurtig opslagsreference.

TABEL A.3: Komponentoversigt for venstre-arm-opsætningen.

Komponent	Antal	Kommentar
Waveshare ST3215-servo	7	12 V, ID 1–7.
Waveshare SC09-servo	5	5 V, håndens fem fingre.
ESP32-baseret USB-bridge	1	USB til servobus.
Mean Well DDR-120	1	DC-DC-omformer, 12 V til ST3215.
Chinfa DRD15 13.5W	1	DC-DC-omformer, 5 V til SC09.
RS Pro 264V 120W	1	Hovedforsyning, 120 W.
Siemens Sentron 8A	2	Sikringer på 12 V- og 5 V-skinnen.
USB-hub med 3 porte	1	Rød, gul, hvid.
Jetson Orin Nano	1	ROS 2-host.

A.4 Figur 1 – Step-respons for skulderens pitch-led

Figur A.1 viser et repræsentativt step-respons-forløb for skulderens pitch-led. Den blå kurve er den kommanderede vinkel som step-funktion, og den orange kurve er den loggede feedback-vinkel. De beregnede metrikker fra Tabel A.2 er markeret direkte på plottet.

FIGUR A.1: Step-respons for skulderens pitch-led ved et step på $\Delta\theta = \text{TODO}^\circ$. Målt under idriftsættelsen.

A.5 Foto-bilag

FIGUR A.2: USB-hub med de tre porte mærket efter kabelfarverne på de tilsluttede ESP32-bokse.

FIGUR A.3: Venstre arm i neutral pose, hvor alle led står på deres default-vinkel fra Tabel A.1.

FIGUR A.4: Venstre arm i strakt pose som brugt under afprøvning af de mekaniske grænser.

FIGUR A.5: ESP32-boksen ("rød boks"), der driver venstre arms servobus.

FIGUR A.6: Launcher-GUI under demo. Service-knapper er placeret øverst, og animations-fanen findes nederst.